

Design Pattern Application and Utility

By Sabbir, AKM

Outline

- Definition
- Back ground of design pattern
- Why we need pattern
- Categories
- Example on categories
- Why modular development
- Conclusion

Definition

- In software development design pattern is a written document that describe a solution to a design problem that repeatedly re-occur in different software development project.
- It's a reusable solution to commonly occurring problem.
- A successful pattern must have applied successfully in previous projects.

Back Ground of Pattern

- Alexander Christopher a civil engineer.
- Kent Beck and Ward Cunningham experimented with the idea of applying this
- It was first gained popularity in CS after the publication of book named “Design-pattern: Elements of reusable object oriented software”

Why we need it

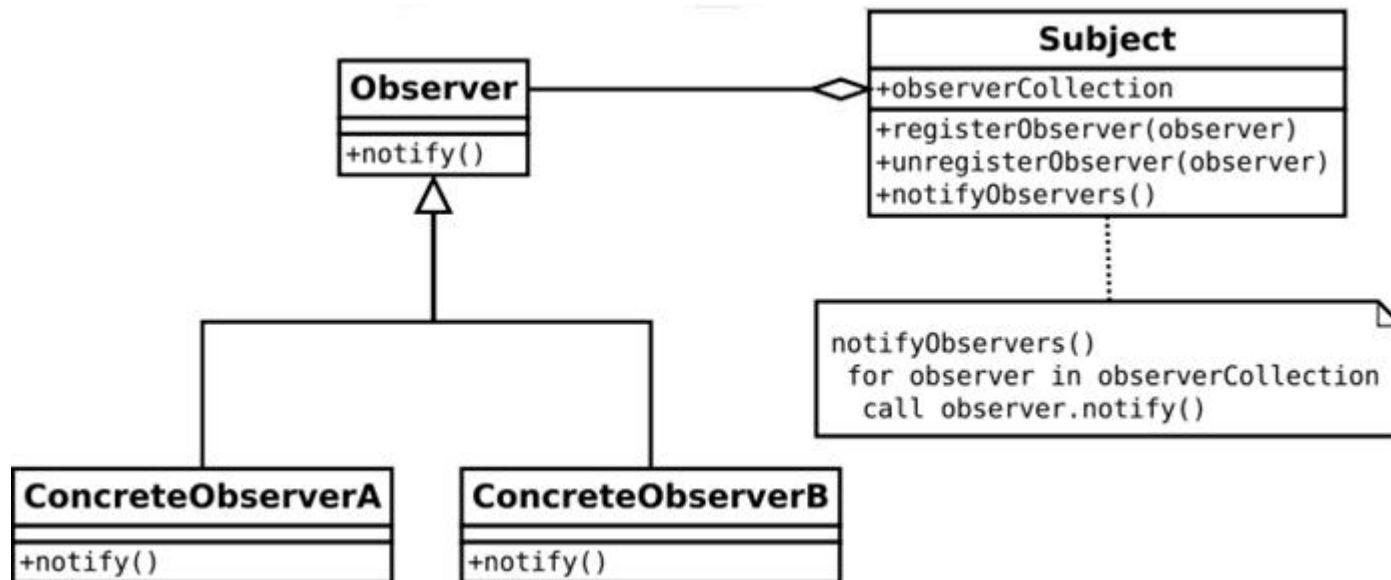
- To design a new software system quickly and efficiently with minimum effort
- Easier to understand existing software system
- “if you have only hammer everything looks like nail”

Types of Patterns

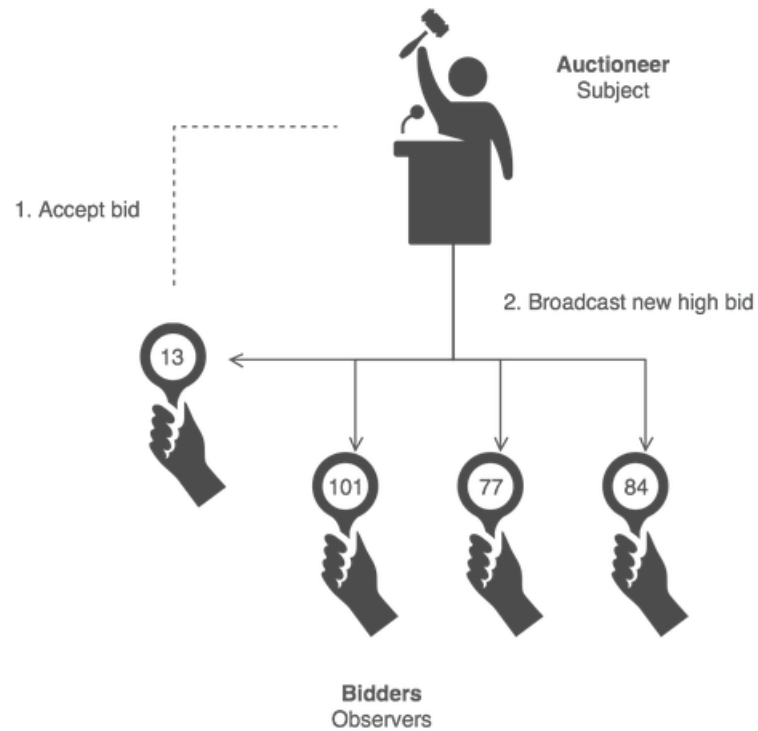
- Creational Patterns
 - Objects creation
- Structural Patterns
 - Relationship between two objects or entities
- Behavioral Design Patterns
 - Communication between two objects of same type or different type

Observer Pattern

- Observer design pattern is a software design pattern in which an object known as subject maintains a list of dependent objects. Called observers and notifies them automatically any of the state changes, usually by calling one of their methods
- Type: behavioral pattern
- It defines the one to many relationship



Observer Pattern



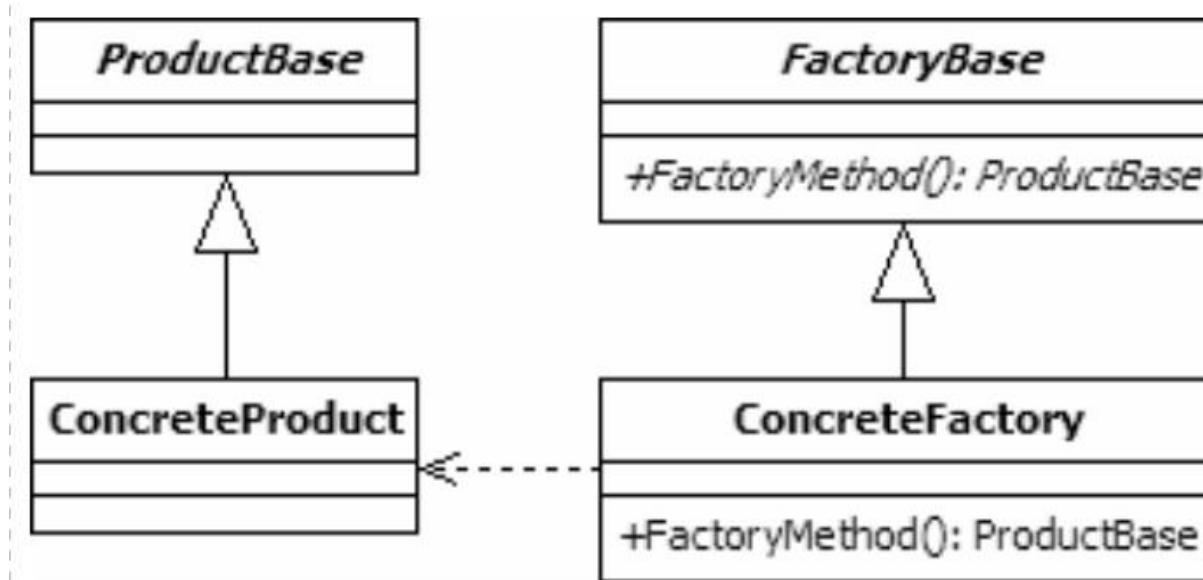
Creational Patterns

- Creational pattern deals with the problem of object creation mechanism
- The basic form of object creation can result in design problems and add complexity to the system
- There are several creational patterns such as
 - Factory
 - Builder
 - Abstract factory
 - Singleton pattern

Factory Pattern

- Its creational pattern
- It control the object instantiation
- It abstract the object generation process so that the type of object need to be generated can be determined at runtime
- Unnecessary mem allocation, resource leakage, code duplication

Factory Pattern

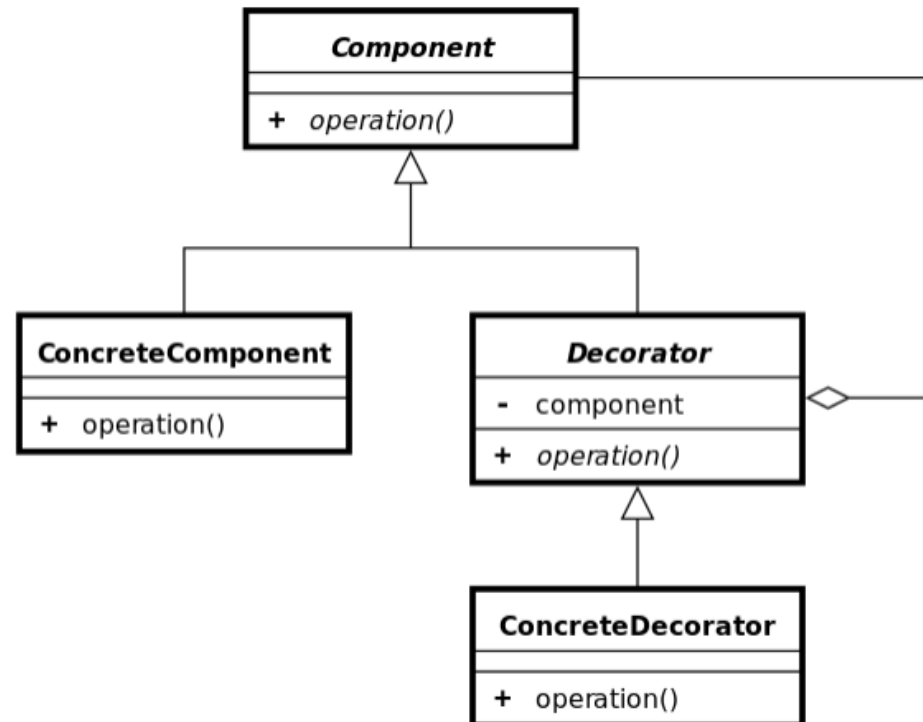


Structural Pattern

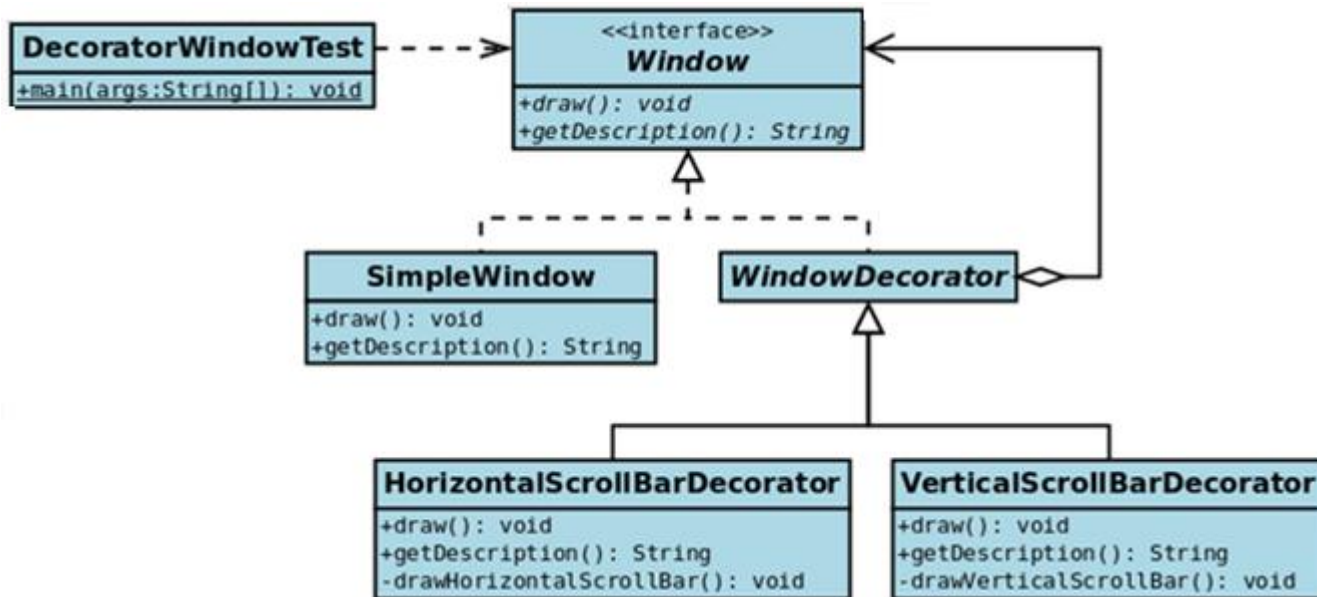
- It identifies the simple way to realize the relationship between different objects
 - Adapter
 - Decorator
 - Bridge
 - Façade
 - Flyweight

Decorator Pattern

- we use to extend the certain functionality of object during runtime independent of other instances



Decorator pattern



Modular development

- Design pattern helps to implement modularize development
- Different components developed independently
- Compile each component independently
- Search for error is easier
- Reusable module
- Quickly and easily assembled by large team

Conclusion

- Software component design can be fun
- Design pattern seems easy to understand but its not
- It may slow down the development process but have long run negative effect